

Documento Técnico: Paquete C - Motor de Cumplimiento y Panel de Monitoreo

1. Responsabilidad y Alcance

El Paquete C constituye el subsistema de evaluación de riesgos y auditoría de cumplimiento (Compliance) dentro de la Plataforma OSINT Ecuador. Su propósito principal es procesar los datos de inteligencia obtenidos, evaluarlos contra bases de datos de listas restrictivas internacionales (como OpenSanctions) y generar alertas persistentes en tiempo real para el equipo de análisis.

2. Arquitectura y Stack Tecnológico

El componente está segmentado en dos capas principales:

- **Backend (Compliance Engine):**
 - **Framework:** Java 17 nativo y Spring Boot 3.x.
 - **Base de Datos Relacional:** PostgreSQL 15, encargada de la persistencia transaccional y estructurada de las alertas de riesgo.
 - **Base de Datos Documental:** Elasticsearch 8, utilizada para la indexación profunda y búsquedas de texto completo sobre los reportes generados.
 - **Mensajería Asíncrona:** RabbitMQ, empleado para la coreografía de eventos.
- **Frontend (Dashboard Monitor):**
 - **Framework:** React.js con Vite.
 - **Estilización:** Tailwind CSS v4 y PostCSS.
 - **Arquitectura UI:** Single Page Application (SPA) orientada al monitoreo en tiempo real con diseño basado en Glassmorphism.

3. Contratos de Integración Basados en Eventos

El sistema implementa una arquitectura orientada a eventos (EDA) utilizando el exchange de enrutamiento `osint.exchange`.

3.1. Eventos Consumidos (Input)

- **Clave de Enrutamiento (Routing Key):** `osint.completado`
- **Cola de Mensajes:** `compliance.osint.completado.queue`
- **Funcionalidad:** El sistema se suscribe a la finalización de los reportes OSINT emitidos por el Paquete A. El payload contiene los datos extraídos y el enlace al documento consolidado.
- **Estructura del Payload Esperado (JSON):**

```
{
  "requestId": "UUID",
  "targetId": "UUID",
  "pdfUrl": "s3://ruta/reporte.pdf",
  "status": "COMPLETED",
  "completedAt": 1715000000000,
  "data": {
    "fullName": "NOMBRE COMPLETO",
    "birthDate": "YYYY-MM-DD",
    "civilStatus": "Soltero/Casado"
  }
}
```

3.2. Eventos Emitidos (Output)

- **Clave de Enrutamiento (Routing Key):** `alerta.compliance`
- **Funcionalidad:** Tras la evaluación en el motor de riesgos, si la probabilidad de coincidencia (score) supera el umbral estricto del 80% (0.8), el sistema persiste el hallazgo y notifica al bus de mensajes para que los demás dominios de la arquitectura tengan visibilidad de la alerta.
- **Estructura del Payload Emitido (JSON):**

```
{
  "alertId": "UUID",
  "requestId": "UUID",
  "targetId": "UUID",
  "fullName": "NOMBRE COMPLETO",
  "riskLevel": "HIGH",
  "matchReason": "Razón del match con la base de datos externa",
  "timestamp": 1715000000000
}
```

4. Especificación de Endpoints (API REST)

El servicio expone los siguientes endpoints (desplegados localmente en el puerto 8081) para consumo exclusivo de la capa de presentación:

- GET /api/v1/compliance/alerts: Recupera un arreglo estructurado con todas las alertas históricas, ordenadas de manera descendente según su marca de tiempo.
- GET /api/v1/compliance/search?q={criterio}: Ejecuta una consulta directa sobre el clúster de Elasticsearch para recuperar reportes indexados.

5. Procedimiento de Despliegue Local

Para ejecutar el entorno en una estación de trabajo de desarrollo, se deben seguir los siguientes lineamientos:

5.1. Inicialización de Infraestructura

Se requiere el despliegue de los contenedores de soporte mediante Docker Compose desde el directorio raíz del proyecto:

```
docker-compose up -d app4-rabbitmq app3-postgres app3-elasticsearch
```

(Nota Técnica: PostgreSQL se aprovisiona en el puerto 5433 bajo las credenciales `compliance_user` / `compliance_password`).

5.2. Despliegue del Backend

Desde el directorio `app3-compliance/backend`, se ejecuta el motor de Spring Boot sin dependencias de Lombok para evitar conflictos de compilación en versiones modernas del JDK:

```
./mvnw clean spring-boot:run
```

5.3. Despliegue del Frontend

Desde el directorio `app3-compliance/frontend`, se compilan y levantan los recursos web:

```
npm run dev
```

6. Pruebas de Integración y Entorno Mock

Para facilitar las pruebas de desarrollo sin incurrir en costos asociados al consumo de APIs externas reales (OpenSanctions), el componente incorpora un servicio simulado denominado `OpenSanctionsMockService`.

El comportamiento de este mock es determinista: si el evento entrante en RabbitMQ contiene la cadena "PEREZ" o "SANCIONADO" en el atributo `fullName`, el sistema calcula una probabilidad de riesgo del 89% (0.89), desencadenando exitosamente la generación de la alerta en PostgreSQL y su respectiva visualización en el Panel de Monitoreo. Para cualquier otra cadena, el sistema asigna un riesgo nominal y omite la alerta.